

Proyecto Final Aprendizaje Reforzado

Condiciones de armado de los grupos:

Se permiten grupos de hasta 3 estudiantes, de manera excluyente.

Tema:

Se debe elegir uno de los 3 tracks propuestos. Dentro de cada track, el tema puede ser uno de los sugeridos por la cátedra, o bien un tema diferente acordado con los profesores, con objetivos a coordinar con ellos. Una vez convenidos los objetivos mínimos del trabajo, deberán cumplirse sin excepción. Eventualmente, por iniciativa propia o por acuerdo con los profesores, el trabajo podrá extenderse más allá del mínimo establecido.

Fechas de entrega:

- 01/07 23:59: Entrega del informe escrito, junto con el código y el material complementario pertinente.
- (Fecha aproximada!! 08/07): Defensa oral de 15 minutos, en la cual los profesores harán preguntas a cada miembro del grupo sobre el informe entregado y asignarán la nota final.

Informe y Entregable:

Se deberá presentar un informe de no más de 10 páginas, exponiendo las técnicas y decisiones tomadas a lo largo del trabajo. Se recomienda que el informe tenga una estructura similar a la siguiente:

- Objetivos: descripción del tema de trabajo y de sus objetivos particulares, con una extensión aproximada de media página.
- Introducción: background necesario para comprender el tema y toda la información resumida que resulte relevante para entender el desarrollo del trabajo, incluida la especificación del ambiente de aprendizaje. Extensión sugerida: entre media página y una página.
- Desarrollo: historia cronológica del desarrollo del trabajo, incluyendo los dead ends, problemas de implementación, experimentación con distintos parámetros de entrenamiento, limitaciones del algoritmos en el entorno elegido, etc.
- Conclusiones, puntos importantes del desarrollo, principales resultados y posibles trabajos futuros.

El informe puede incluir links a drive con videos donde se muestran los comportamientos aprendidos por el agente.

Particularidades:

- Para trabajos de implementación de un algoritmo de RL conocido, se debe explicar qué logra cada una de las optimizaciones implementadas, mostrando las partes del código correspondientes. También se pide probar el algoritmo sobre todos los probe environments y comparar la performance del algoritmo con alguna implementación de referencia (por ejemplo stable baselines 3) sobre cart pole, y finalmente la performance del algoritmo en el ambiente elegido.
- Para trabajos que usen un algoritmo de RL ya programado, se pide hacer un informe del ambiente (si fue desarrollado para el trabajo), y de los comportamientos aprendidos por la política. Los objetivos particulares para cada trabajo de este tipo van a ser coordinados con los profesores.

El entregable debe contener:

- Informe.
- Código fuente utilizado para resolver el problema
- Enlaces o referencias a cualquier recurso externo utilizado

Si el entregable supera el límite de espacio del Campus Virtual deberán entregar por el campus únicamente el informe dónde deberá indicar en el texto un enlace a Google Drive con el entregable completo.

Tracks de trabajos finales:

A. Implementar un algoritmo de RL conocido desde 0

Las siguiente consigas son las más guiadas, no pueden usar ninguna librería con algoritmos ya programados de RL como los disponibles en [Stable Baselines3](#), pero si librerías de diferenciación automática como Pytorch, JAX o tensorflow

PPO-C, SAC, TRPO, [Rainbow](#) o un algoritmo similar y resuelva un ambiente conocido como:

- <https://gymnasium.farama.org/environments/box2d/>
- <https://pypi.org/project/gym-sailing/>
- <https://github.com/markub3327/flappy-bird-gymnasium>
- <https://ale.farama.org/environments/>
- [MinAtar](#)
- Otro ambiente aprobado por la cátedra (No se recomienda usar ambientes donde la observación sean píxeles crudos por la cantidad de cómputo que esto requiere)
- Programe [RLHF](#) y entrene el ambiente [pendulum](#) con el método.
- (individual) Lea [Learning to Drive a Bicycle using Reinforcement Learning](#) y recree la imagen que está en la tapa del Sutton y Barto

B. Proponer un problema a resolver, programar el ambiente y analizar los comportamientos aprendidos. Se pueden usar librerías de RL ya implementadas (como SB3, CleanRL, Puffer4, etc.)

Algunos ejemplos propuestos por la cátedra:

- Cree un simulador para: <https://github.com/Gabo-Tor/pendulum> entrene un agente en el usando domain randomization y luego pruébelo en el hardware real.
- Entrene en hardware: <https://github.com/Gabo-Tor/pendulum> y haga un estudio de ablaciones y del efecto de distintos hiperparámetros.
- Creen un agente de RL que mitigue el efecto de [ondas de transito](#) en una ring-road.
- Entrenar un agente que recupere un avión simulado luego de una entrada en pérdida, el ambiente para esto será provisto por la cátedra.

C. Explicar un paper famoso. Para este track, además se pide preparar un póster para la AI Fest. Este track es solo individual:

- Generalización en [Procgen](#): elija un experimento y repita el experimento de generalización siguiendo el paper original.
- Lea [Policy Invariance Under Reward Transformations](#) explíquelo y corra experimentos donde se destaquen los puntos principales de este trabajo.
- Lea [Go-Explore: a New Approach for Hard-Exploration Problems](#) Implemente el algoritmo de exploración y pruébelo en un gridworld.
- Lea [Time Limits in Reinforcement Learning](#) y reproduzca experimentos que muestren los efectos de truncation vs termination y el manejo correcto del tiempo límite.
- Lea [Hindsight Experience Replay](#) explique el método y demuéstrela en Bit-Flipping y en un entorno tipo [FetchReach](#) con recompensa escasa
- Lea [Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents](#) y muestre por qué las Sticky Actions son importantes creando un agente que explote el determinismo del ambiente y comparándolo con un agente de referencia como DQN.
- Lea [A Machine Learning Approach to Visual Perception of Forest Trails for Mobile Robots](#) y cree un agente similar usando el dataset de conducción autónoma de UdeSA.